Object Oriented Programming: Python BootCamp

HIT-emlyon BSc Program
April-May 2024, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 01 : Variables | Data Types

This lecture content will be about Variables and Data Types.

Variables:

In Python, variables are names that can be assigned a value and then used to refer to that value throughout your code.

Variables are fundamental to programming for two reasons:

1. Variables keep values accessible: For example, you can assign the result of some time-consuming operation to a variable so that your program doesn't have to perform the operation each time you need to use the result.

2. Variables give values context: The number 28 could mean lots of different things, such as the number of students in a class, the number of times a user has accessed a website, and so on. Giving the value 28 a name like num_students makes the meaning of the value clear.

In this section, you'll learn how to use variables in your code, as well as some of the conventions Python programmers follow when choosing names for variables.

The Assignment Operator (=):

An operator is a symbol, such as +, that performs an operation on one or more values. For example, the + operator takes two numbers, one to the left of the operator and one to the right, and adds them together.

Values are assigned to variable names using a special symbol called the assignment operator (=). The = operator takes the value to the right of the operator and assigns it to the name on the left.

```python
age = 25
```

```python
print(age)
```

A **function** is code that performs some task and can be invoked by a name.

The above code invokes, or calls, the **print()** function with the variable **age** as input.

The parentheses enclose everything that gets sent to the function as input.

print(age) displays the output 25 because Python looks for the name **age**, finds that it's been assigned the value 25, and replaces the variable name with its value before calling the function.

Variable names are case sensitive, so a variable named **age** is not the same as a variable named **Age**.

For instance, the following code produces a NameError:

```python
print(Age)
```

Rules for Valid Variable Names:

Variable names can be as long or as short as you like, but there are a few rules that you must follow.

Variable names may contain uppercase and lowercase letters (A–Z, a–z), digits (0–9), and underscores (_), but they cannot begin with a digit.

```python
test1 = 1
_alpha = 2
L = 3
```

```python
3text = 4
```

In Python, it's common to write variable names in lower_case_with_underscores.

Leave Yourself Helpful Notes

You can leave comments in your code. Comments are lines of text that don't affect the way a program runs. They document what code does or why the programmer made certain decisions.

How to Write a Comment The most common way to write a comment is to begin a new line in your code with the # character. When you run your code, Python ignores lines starting with #.

```python
# This variable stores the car length in cm
L = 2000
```

```python
print(L)
```

```python
H = 146 # This variable stores the car height in cm
```

```python
print(H)
```

Data Types

The term **data type** refers to what kind of data a value represents. There are several data types built into Python. Such as Numerical data types (integer, float), boolean data type or string data type.

Numerical data type

```python
num = 100          # An integer variable
length = 10.7      # A floating point variable
```

```python
print(num)
```

```python
type(num)
```

```python
type(length)
```

The **type()** function, is used to determine the data type of a given value or a variable.

Boolean data type

The fundamental data type boolean, or bool for short, can have only one of two values. In Python, these values are conveniently named **True** and **False**.

```python
is_finished = True
```

```python
type(is_finished)
```

```python
is_finished = False
```

```python
type(is_finished)
```

We will talk more about boolean values when we introduce conditional expressions.

Strings

Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Strings are one of the fundamental Python data types.

Special functions called **string methods** are used to manipulate strings.

There are string methods for changing a string from lowercase to uppercase, removing whitespace from the beginning or end of a string, replacing parts of a string with different text, and much more.

You can create a string by surrounding some text with quotation marks:

```python
name1 = "Emlyon"
```

Python allows for either pairs of single or double quotes for the strings.

```python
print(name1)
```

```python
type(name1)
```

The string data type has a special abbreviated name in Python: str.

Strings have three important properties:

1. Strings contain individual letters or symbols called characters.
2. Strings have a length, defined as the number of characters the string contains.
3. Characters in a string appear in a sequence, which means that each character has a numbered position in the string

Small note before we continue.

If you try to use double quotes inside a string delimited by double quotes, you'll get an error:

```python
text1 = "She said, "What time is it?""
```

However, you can use a single quote in a string delimited by double quotes, and vice versa:

```python
text1 = 'She said, "What time is it?"'
```

You can also use the escape character i.e. backslash.

```python
text1 = "She said, \"What time is it?\""
```

```python
print(text1)
```

Determine the Length of a String

The number of characters contained in a string, including spaces, is called the length of the string.

```python
text2 = "This is a test string"
```

```python
len(text2)
```

Python built-in function **len()** can be used to determine the length of a string.

Multiline Strings

To deal with long strings, you can break them up across multiple lines into multiline strings.

There are a couple of ways to tackle this. One way is to break the string up across multiple lines and put a backslash () at the end of all but the last line.

```python
long_text = "This planet has—or rather had—a problem, which was \
this: most of the people living on it were unhappy for pretty much of the time."
```

```python
print(long_text)
```

You can also create multiline strings using triple quotes (""" or ''') as delimiters.

```python
long_text = """This planet has—or rather had—a problem, which was
this: most of the people living on it were unhappy for pretty much of the time."""
```

```python
print(long_text)
```

Concatenation, Indexing, and Slicing

Concatenation, which joins two strings together

You can combine, or concatenate, two strings using the + operator:

```python
string1 = "Hello"
string2 = "World"
string3 = string1 + string2
```

```python
print(string3)
```

Indexing, which gets a single character from a string

Each character in a string has a numbered position called an index.

You can access the character at the **nth** position by putting the number **n** between two square brackets [] immediately after the string name.

```python
text = "Python BootCamp"
```

```python
text[7]
```

In Python—and in most other programming languages counting always starts at zero 0.

Strings also support negative indices

```python
text[-1]
```

Slicing, which gets several characters from a string at once

You can extract a portion of a string, called a substring, by inserting a colon between two index numbers set inside square brackets like this:

```python
text[0:6]
```

text[0:6] returns the first six characters of the string assigned to **text**, starting with the character at index 0 and going up to but not including the character at index 6.

The **[0:6]** part of **text[0:6]** is called a **slice**.

If you omit the first index in a slice, then Python assumes you want to start at index 0:

```python
text[:6]
```

Similarly, if you omit the second index in the slice, then Python assumes you want to return the substring that begins with the character whose index is the first number in the slice and ends with the last character in the string:

```python
text[7:]
```

Strings Are Immutable

Strings are immutable, which means that you can't change them once you've created them.

If you try to assign a new letter to one particular character of a string, Python throws a TypeError and tells you that str objects don't support item assignment.

```python
text[6] = '_'
```

String Methods

Strings come bundled with special functions called string methods that you can use to work with and manipulate strings. There are numerous string methods available, but we'll focus on some of the most commonly used ones.

Converting String Case

```python
"Python BootCamp".lower()
```

The dot (.) tells Python that what follows is the name of a method.

Removing Whitespace

```python
"  Python BootCamp".strip()
```

Counting

Want to know how often one string occurs in another? You can just use count():

```python
"  Python BootCamp is the best Python program ever!".count("Python")
```

Find

You can use find() to get the index of the first occurence of one string in another. If there is none, it will return -1.

```python
"  Python BootCamp is the best Python program ever!".find("python")
```

Starts and Ends

```python
"Python BootCamp".startswith('P')
```

String Methods and Immutability

Recall from the previous section that strings are immutable —they can't be changed once they've been created.

Most string methods that alter a string, like .strip() and .lower(), actually return copies of the original string with the appropriate modifications.

```python
name = "Emlyon"
```

```python
name.upper()
```

```python
name
```

When you call name.upper(), nothing about name actually changes.
If you need to keep the result, then you need to assign it to a variable:

```python
name = name.upper()
```

```python
name
```

More on String Methods:

Here is a set of built-in methods that you can use on strings. We will see more of this throughout the course.

None Data Type

The `None` keyword is used to define a null value, or no value at all.

None is not the same as 0, False, or an empty string. None is a data type of its own (NoneType) and only None can be None.

```python
my_var = None
print(my_var)
```

Do's:

- Do create variables with meaningful names
- Do use underscores _ for longer variables names, like **long_variable_name**
- Do start your variable names with a lower case letter

Dont's:

- Don't call a variable "print", "for", "in" etc. since these are build in functions of python that you can override
- Don't create a variable that starts with a number, like **01_var**
- Don't create too long variable names

Arithmetic Operators

Operators are used to perfrom operations on variables and values. Different variable types require a set of different operations (sometimes!)

You could use arithmetic operators on integers and float type (such as below):

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponent
//	Floor division

```python
2 + 2
```

```python
3 * 2
```

```python
3 ** 2
```

In Python 3, `/` denotes the standard division and `//` denotes the integer division, whereas in Python 2, `/` denotes the integer division.

```python
5 / 2
```

```python
5 // 2
```

You may also use some of the arithmetic operations over 'str' type, for example:

```python
'2' + '4'
```

```python
'2' * '4'
```

```python
2 * 'S'
```

Comparison Operators

The result of the 'comparison' is a boolean value (**True** or **False**)

Operator	Description	Example
==	equal	1 == 1
!=	not equal	2.3 != 2.313
<	less than	2.5 < 9
>	greater than	2.4 > 2.399
<=	less than or equal	3 <= 3
>=	greater than or equal	2.4 >= 2.399

```python
2.5 < 3
```

Logical Operators

Logical operators take boolean values as input and produce either **True** or **False** as output.

Operator	Description

x **or** y | if either of x or y is True, return True; otherwise, return False x **and** y | if both x and y are True, return True; otherwise, return False **not** x | reverse x and return (return False, if x is True)

```python
(2 == 3) or (4 < 5)
```

```python
(2 == 3) and (4 < 5)
```

```python
not (2 == 3)
```

We will see more on Comparison and Logical Operators later.

Type casting

If you want to change one data type to another we may use the following functions: int(), str(), float(), bool()

```python
int(9.9)
```

```python
str(5)
```

```python
bool(1)
```

```python
int("Yes!")
```

remember You may need to cast 'str' to 'int' before doing any arithmetic operations:

```python
# wrong
a = 34
b = "10"
a + b
```

```python
# correct
a = 34
b = "10"
a + int(b)
```

Try to see if you could give an argument to the 'bool' function and get a "False" output.

Almost any value is evaluated to True if it has some sort of content. Any string is True, except empty strings. Any number is True, except 0.

These four data types (int,float,str,bool) are so-called "Primitive types".

String Formatting

Suppose you have a string name = "Doctor Strange" and two integers heads = 2 and arms = 3.

You want to display them in the following line: Doctor Strange has 2 heads and 3 arms.

The first way of doing this is using commas to insert spaces between each part of the string inside of a print() function:

```python
name = "Doctor Strange"
heads = 2
arms = 3

print(name, "has", str(heads), "heads and", str(arms), "arms.")
```

Another way to do this is by concatenating the strings with the + operator:

```python
print(name + " has " + str(heads) + " heads and " + str(arms) + " arms.")
```

There's a third way of combining strings: **formatted string literals**, more commonly known as **f-strings**.

Here's what the above string looks like when written as an f-string:

```python
print(f"{name} has {heads} heads and {arms} arms.")
```

There are two important things to notice about the above examples:

1. The string literal starts with the letter f before the opening quotation mark
2. Variable names surrounded by curly braces { and } are replaced with their corresponding values without using str()

f-strings are only available in Python version 3.6 and above.

the **format()** method:

```python
"{} has {} heads and {} arms.".format(name, heads, arms)
```

the **%** operator:

```python
"%s has %s heads and %s arms." % (name, heads, arms)
```

User Inputs:

In this section, you will learn how to get some input from a user with the **input()** function.

Try this:

```python
input()
```

The input() returns any text entered by the user as a **string**.

To make input() a bit more user friendly, you can give it a prompt to display to the user.

The prompt is just a string that you put in between the parentheses of input().

The input() function displays the prompt and waits for the user to type something on their keyboard.

When the user hits Enter, input() returns their input as a string that can be assigned to a variable and used to do something in your program:

```python
age = input("Please enter you age?")
```

```python
print(age)
```

EXERCISES 01 - PART ONE
